

UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Faculty of Engineering, Built Environment and Information Technology

EBB 320

Control Systems

Practical 3: PID Controller Design

Name and Surname	Student Number	Signature	% Contribution
Hanré Coetzer	21507334		33
Cornelis Claasen	24704611		33
B.J. Mncube	23700816		33

By signing this assignment we confirm that we have read and are aware of the University of Pretoria's policy on academic dishonesty and plagiarism and we declare that the work submitted in this assignment is our own as delimited by the mentioned policies. We explicitly declare that no parts of this assignment have been copied from current or previous students' work or any other sources (including the internet), whether copyrighted or not. We understand that we will be subjected to disciplinary actions should it be found that the work we submit here does not comply with the said policies.

October 24, 2026

Contents

1 Theoretical Background **2**
1.1 First Order Responses . . . 2
1.2 Second Order Responses . . . 2
1.3 PID Controllers . . . 2

2 Design and Simulation **4**
2.1 PID Controller Design for $G_{11}(s)$ Using the Frequency Design Techniques 4
2.2 PID Controller Design for $G_{22}(s)$ Using the Frequency Design Techniques 6
2.3 Root Locus Technique for G11 . . . 10
2.4 Root Locus Technique for G22 . . . 12
2.5 Simulation Results Discussion . . . 14
2.5.1 Comparison of Controllers Designed for $G_{11}(s)$ . . . 14
2.5.2 Comparison of Controllers Designed for $G_{22}(s)$ . . . 15

3 Results **16**
3.1 Root Locus Designs . . . 16

4 Discussion **20**
4.1 Frequency Response Technique . . . 20
4.2 Root Locus Technique . . . 21
4.3 Comparison of PID Coefficients Between Design Methods . . . 21

A Code **24**
A.1 Matlab Code . . . 24
A.2 Model Fit Code . . . 38
A.3 Prac 2 — Controller Design (P and PI) . . . 41
A.4 Prac 2 — Report Figure Generation (P and PI) . . . 41
A.5 MCU Code . . . 41

1 Theoretical Background

1.1 First Order Responses

To find the rise time of a First Order Plus Dead Time (FOPDT) transfer function the following equation can be used: [1]

$$T_r = \frac{2.2}{a} \tag{1}$$

The dead time of the FOPDT transfer function can be approximated using Padé first order approximation:

$$e^{-\theta s} \approx \frac{1 - \frac{\theta}{2}s}{1 + \frac{\theta}{2}s} \tag{2}$$

1.2 Second Order Responses

The equation defining a standard second order response is defined as:

$$G(s) = \frac{{\omega_n}^2}{s^2 + 2\zeta\omega_n s + {\omega_n}^2}$$

The equations that then define the dampening ratio and natural radian frequency of the second order response using a pre-define overshoot and rise time are found to be: [1]

$$\zeta = \frac{-\ln(\%\mathrm{OS}/100)}{\sqrt{\pi^2 + \ln^2(\%\mathrm{OS}/100)}} \tag{3}$$

$$\begin{aligned} \omega_n T_r &= 1.76\zeta^3 - 0.417\zeta^2 + 1.039\zeta + 1 \\ \omega_n &= (1.76\zeta^3 - 0.417\zeta^2 + 1.039\zeta + 1)/T_r \end{aligned} \tag{4}$$

The equation defining the natural frequency using the system's rise time is found using curve fitting, as a precise analytical equation is not available. [1]

A design point associated with the found dampening ration and natural radian frequency can then be found using: [1]

$$\sigma_d = \zeta\omega_n \tag{5}$$

$$\omega_d = \omega_n\sqrt{1 - \zeta^2} \tag{6}$$

1.3 PID Controllers

From [1] the basic equations that define a PID controller's transfer function are:

$$G_c(s) = K_p + \frac{K_i}{s} + K_d s = \frac{K_c(s + z_1)(s + z_2)}{s} \tag{7}$$

Therefore, it can be found that:

$$K_p = K_c(z_1 + z_2) \tag{8}$$
$$K_i = K_c(z_1 z_2) \tag{9}$$
$$K_d = K_c \tag{10}$$

To find the parameters of the PID controller the following equations must be satisfied in order to ensure system stability.

The angle equation:

$$\sum \text{zero angles} - \sum \text{pole angles} = \pm 180^\circ \tag{11}$$

The magnitude equation:

$$|K_c G_c(s_d) G_p(s_d)| = 1 \tag{12}$$

2 Design and Simulation

The following table contains the derived transfer functions that were used to simulate and implement the controllers.

Table 1: 2x2 Process Transfer Function Matrix

	Input 1 (U_1)	**Input 2 (U_2)**
Output 1 (Y_1)	$G_{11}(s) = \frac{0.1179}{s+0.0076}e^{-0.145s}$	$G_{12}(s) = \frac{0.03658}{s+0.005254}e^{-0.145s}$
Output 2 (Y_2)	$G_{21}(s) = \frac{0.06368}{s+0.006012}e^{-0.145s}$	$G_{22}(s) = \frac{0.05872}{s+0.005984}e^{-0.145s}$

2.1 PID Controller Design for $G_{11}(s)$ Using the Frequency Design Techniques

The design specifications are a 10% overshoot and a rise time of $T_r = 96.1$ s. The damping ratio ζ is calculated as:

$$\zeta = \frac{-\ln(0.1)}{\sqrt{\pi^2 + [\ln(0.1)]^2}} = 0.591$$

The required natural frequency ω_n is:

$$\omega_n = \frac{1.76(0.591)^3 - 0.417(0.591)^2 + 1.039(0.591) + 1}{96.1} = 0.01906 \text{ rad/s}$$

The required closed-loop bandwidth ω_{BW} and phase margin Φ_M are:

$$\omega_{BW} = \omega_n\sqrt{(1 - 2\zeta^2) + \sqrt{4\zeta^4 - 4\zeta^2 + 2}} = 0.02211 \text{ rad/s}$$

$$\Phi_M = \tan^{-1}\left(\frac{2\zeta}{\sqrt{-2\zeta^2 + \sqrt{1 + 4\zeta^4}}}\right) = 58.58^\circ$$

Following the prescribed procedure, the PI controller zero is fixed at $z_2 = 0.1$, yielding $G_{PI}(s) = \frac{s+0.1}{s}$. The new phase-margin frequency ω_s is selected as 80% of the bandwidth:

$$\omega_s = 0.8 \times 0.02211 = 0.01769 \text{ rad/s}$$

The target total loop phase at ω_s is:

$$\phi_{required}(\omega_s) = -180^\circ + 58.58^\circ = -121.42^\circ$$

The phase contributions of the uncompensated plant (including dead time) and the PI controller at ω_s are:

$$\phi_G(0.01769) = -0.07^\circ - 66.77^\circ = -66.84^\circ$$
$$\phi_{PI}(0.01769) = -90^\circ + \tan^{-1}\left(\frac{0.01769}{0.1}\right) = -79.97^\circ$$

The required phase addition from the D-action zero is:

$$\phi_{add}(\omega_s) = -121.42^\circ - (-66.84^\circ) - (-79.97^\circ) = +25.39^\circ$$

The D-action zero z_1 is located to provide exactly 25.39° of phase:

$$\tan(25.39^\circ) = \frac{0.01769}{z_1} \implies z_1 = 0.03727$$

The controller gain K_{cPID} is calculated to force unity magnitude at ω_s:

$$|G_{11}(j0.01769)| \times \frac{K_{cPID}|j0.01769 + 0.03727||j0.01769 + 0.1|}{|j0.01769|} = 1$$
$$6.121 \times K_{cPID} \times \frac{0.04126 \times 0.10155}{0.01769} = 1 \implies K_{cPID} = 0.6901$$

Expanding the controller transfer function yields the parallel PID gains:

$$G_{c11}(s) = 0.6901\frac{(s + 0.03727)(s + 0.1)}{s} = 0.6901s + 0.0947 + \frac{0.00257}{s}$$

Final Gains for G_{11}: $K_p = 0.0947$, $K_i = 0.00257$, $K_d = 0.6901$

To confirm the design, the bode plot of the open-loop system was plotted and the crossover frequency was compared to the designed crossover frequency.

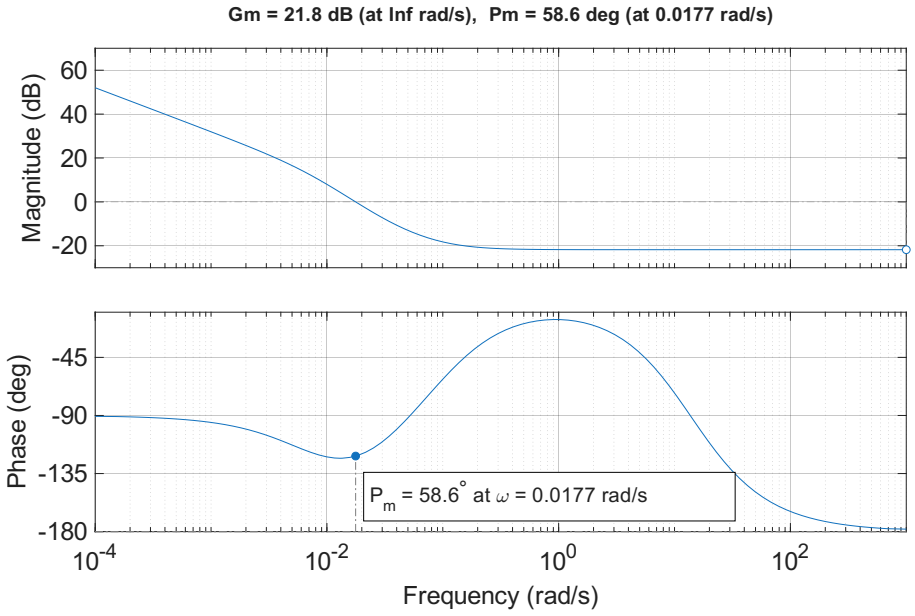


Figure 1: G11 Open-Loop Bode Plot verifying the 58.6° Phase Margin at 0.0177 rad/s.

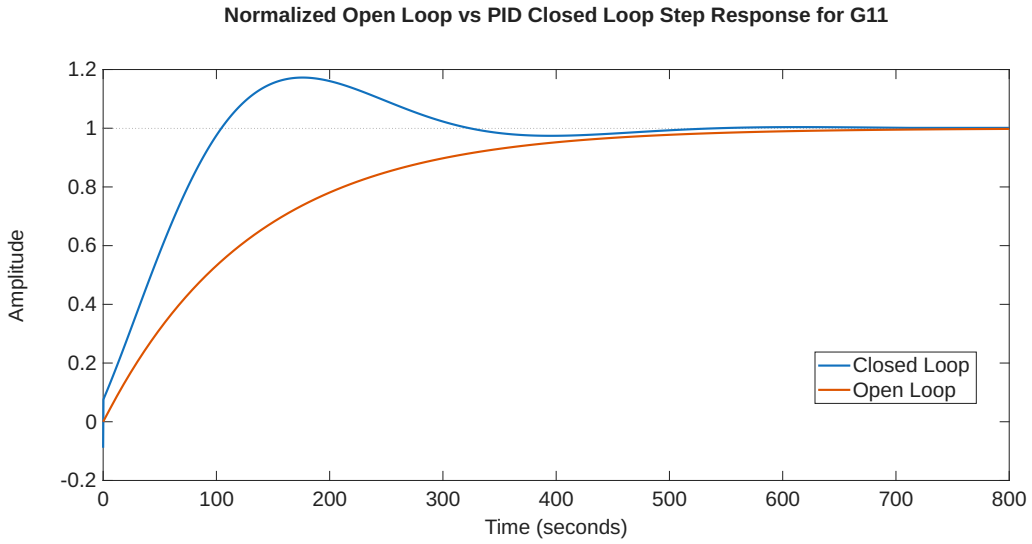


Figure 2: G11 Normalized Open-Loop vs Closed-Loop Step Response.

Figure 2 compares the step response of the open-loop and closed-loop system. It can be observed that the closed-loop system yielded a faster rise time compared to the open loop system.

The following table compares the step response information of the closed-loop and open-loop system.

Table 2: Table comparing open vs closed loop step response characteristics of G11

Step Response Parameter	Open Loop	Closed Loop
Rise Time (s)	289.08	85.29
Transient Time (s)	514.89	432.80
Settling Time (s)	514.89	441.73
Overshoot (%)	0.00	17.26
Undershoot (%)	0.00	8.86
Peak Time (s)	963.45	176.04

2.2 PID Controller Design for $G_{22}(s)$ Using the Frequency Design Techniques

For a target rise time of $T_r = 122.55$ s and identical 10% overshoot ($\zeta = 0.591$, $\Phi_M = 58.58°$), the bandwidth requirements shift:

$$\omega_n = \frac{1.8316}{122.55} = 0.01495 \text{ rad/s}$$
$$\omega_{BW} = 0.01495 \times 1.160 = 0.01734 \text{ rad/s}$$

With $z_2 = 0.1$, the new phase-margin frequency is:

$$\omega_s = 0.8 \times 0.01734 = 0.01387 \text{ rad/s}$$

The required total loop phase is identical: $\phi_{required} = -121.42°$. Evaluating the G_{22} plant and PI controller at the new ω_s:

$$\phi_G(0.01387) = -66.61°(\text{Core}) - 0.12°(\text{Delay}) = -66.73°$$
$$\phi_{PI}(0.01387) = -90° + \tan^{-1}\left(\frac{0.01387}{0.1}\right) = -82.10°$$

The required phase addition is:

$$\phi_{add}(\omega_s) = -121.42° - (-66.73°) - (-82.10°) = +27.41°$$

The D-action zero z_1 is located to provide 27.41° of phase:

$$\tan(27.41°) = \frac{0.01387}{z_1} \implies z_1 = 0.02674$$

The overall controller gain K_{cPID} is found via unity magnitude mapping:

$$|G_{22}(j0.01387)| \times \frac{K_{cPID}|j0.01387 + 0.02674||j0.01387 + 0.1|}{|j0.01387|} = 1$$
$$3.885 \times K_{cPID} \times \frac{0.03013 \times 0.10096}{0.01387} = 1 \implies K_{cPID} = 1.174$$

Expanding the controller polynomial yields:

$$G_{c22}(s) = 1.174\frac{(s + 0.02674)(s + 0.1)}{s} = 1.174s + 0.1488 + \frac{0.00314}{s}$$

Final Gains for G_{22}: $K_p = 0.1488$, $K_i = 0.00314$, $K_d = 1.174$

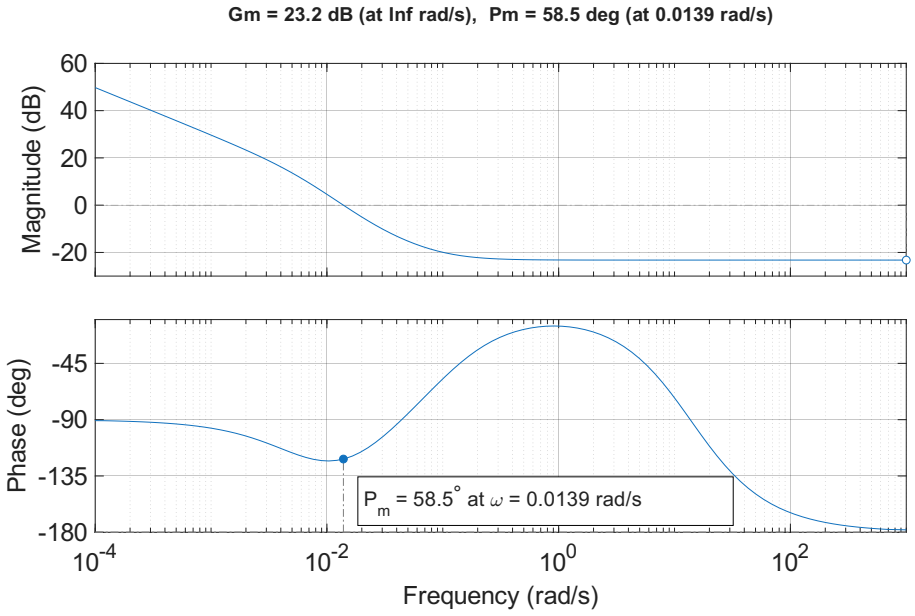


Figure 3: G22 Open-Loop Bode Plot verifying the 58.5° Phase Margin at 0.0139 rad/s.

Figure 3 confirms the designed phase margin of 58.5° at the gain crossover frequency of 0.0139 rad/s. The gain margin is infinite, which is expected for a first-order plant with a PID controller, as the loop phase never reaches −180°. The phase peak visible above −90° is the contribution of the D-action zero z_1, which adds the phase lead required to meet the phase margin specification.

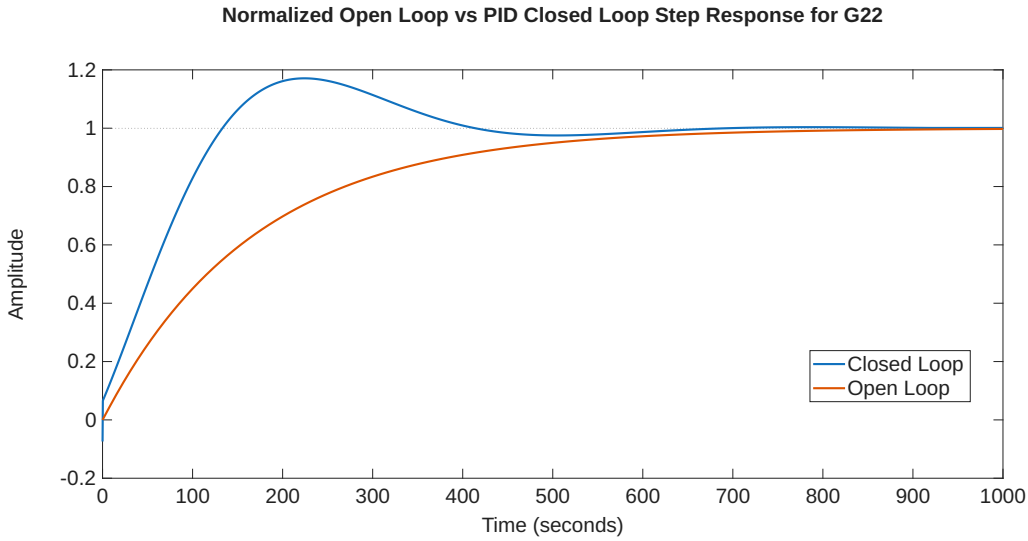


Figure 4: G22 Normalized Open-Loop vs Closed-Loop Step Response.

Figure 4 compares the normalised step response of the open-loop plant and the closed-loop system with the CFR22 controller. The closed-loop system achieves a significantly faster rise time relative to the open-loop plant, satisfying the threefold reduction specification. The initial undershoot observed in the closed-loop response is caused by the dead time approximation, which introduces a right-half-plane zero into

the loop. The closed-loop response converges to unity steady-state value, confirming zero steady-state error as provided by the integrator in the PID controller.

Table 3: Table comparing open vs closed loop step response characteristics of G22

Step Response Parameter	Open Loop	Closed Loop
Rise Time (s)	367.15	107.47
Transient Time (s)	653.90	548.75
Settling Time (s)	653.90	559.16
Overshoot (%)	0.00	17.10
Undershoot (%)	0.00	7.40
Peak Time (s)	1223.60	224.25

The closed-loop rise time of 107.47 s represents a 3.42× reduction from the open-loop rise time of 367.15 s, exceeding the required threefold specification. The overshoot of 17.10% is higher than the target of 10%. This is a known limitation of the frequency response design method, as the procedure assumes a second-order closed-loop approximation. The actual closed-loop is higher order due to the PID zeros, which increase the overshoot beyond what the damping ratio alone predicts. The zero steady-state error is confirmed by the DC gain of unity, achieved through the integrator in the PID controller.

2.3 Root Locus Technique for G11

For the transfer function G_{11} using equation [1] the current rise time was found to be ≈ 289.78 sec. Therefore, the target rise time is 96.593 sec.

Then using equations [3] and [4] it was found that:

$$\begin{aligned}\zeta &\approx 0.5912\\ \omega_n &\approx 0.018967\end{aligned}$$

The design point was then found using [5] and [6] giving $s_d = -0.01121 + j0.0153$.

The second form of the PID equation [7] was then used to find the controller transfer function.

Furthermore, the Padé approximation [2] was used to find an approximation of the transfer function's dead time. The new plant transfer function was then found to be:

$$G_p(s) = \frac{(1 - 0.0725s)}{(1 + 0.0725s)} \frac{0.1175}{(s + 0.007592)}$$

Letting $z_2 = 0.1$ the angles of the poles and zeros to the design point, and [11], it was found:

$$\begin{aligned}&\theta_{z=z_1} + \theta_{z=z_2} + \theta_{z=13.793} - (\theta_{p=0} + \theta_{p=-13.793} + \theta_{p=-0.007592}) = \pm 180^\circ\\ &\theta_{z=z_1} \approx -180^\circ + 219.90^\circ\\ &\theta_{z=z_1} \approx 39.905^\circ\end{aligned}$$

Therefore, the value of z_1 was found as:

$$\begin{aligned}\tan(\theta_{z=z_1}) &= \frac{0.0153}{z_1 - 0.01121}\\ z_1 &= \frac{0.0153 + 0.01121\tan(\theta_{z=z_1})}{\tan(\theta_{z=z_1})}\\ z_1 &\approx 0.0295055\end{aligned}$$

The controller gain K_c was found using [12] where:

$$\begin{aligned}|K_c G_c(s_d) G_p(s_d)| &= 1\\ |G_c(s_d) G_p(s_d)| &\approx 0.848\end{aligned}$$

Therefore:

$$K_c = \frac{1}{|G_c(s_d) G_p(s_d)|} \approx 1.17921$$

The final controller transfer function was then found to be:

$$G_c(s) = \frac{1.17921(s + 0.1)(s + 0.0295055)}{s}$$

Using [10], [9] and [8] the PID values where found to be:

$$K_p = K_c(z_1 + z_2) = 0.152714$$
$$K_i = K_c(z_1 z_2) = 0.003479$$
$$K_d = K_c = 1.17921$$

In order to confirm the design, the root locus was made using matlab, where the root locus plot has an intersection very close to the design point.

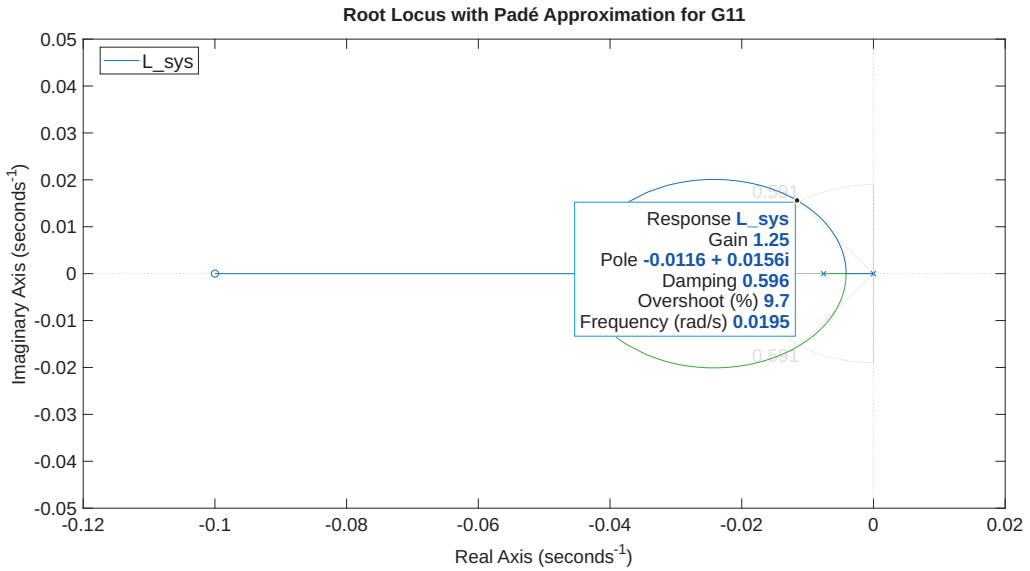


Figure 5: Root Locus plot of G11 with PID controller showing the intersection with the design point.

The resulting time domain comparison of the new closed loop transfer function and the open loop transfer function are shown below:

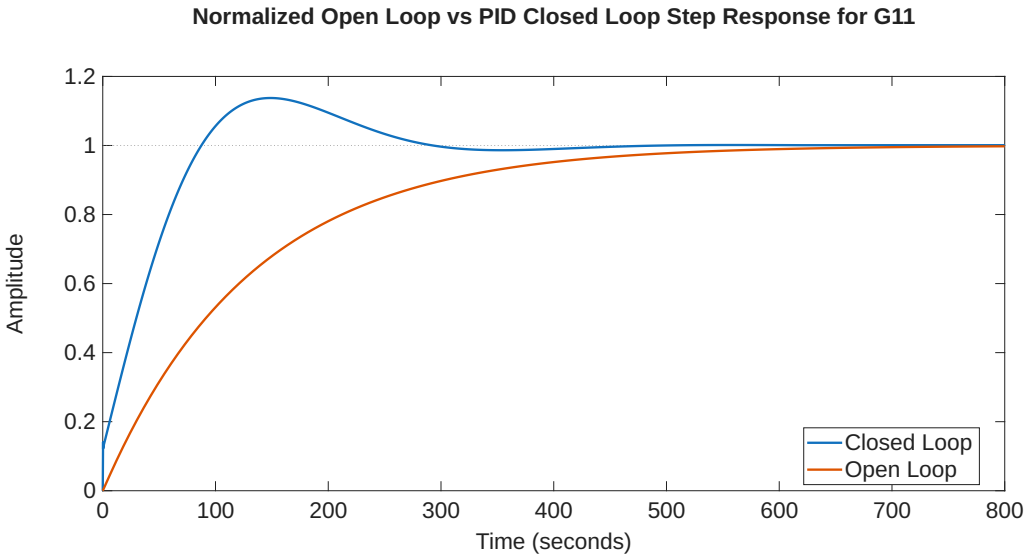


Figure 6: Time domain comparison of open loop plant transfer function against the closed loop system transfer function for G11

The following table compares the open loop and closed loop step response information.

Table 4: Table comparing open vs closed loop step response characteristics of G11

Step Response Parameter	Open Loop	Closed Loop
Rise Time (s)	289.39	71.09
Transient Time (s)	515.43	263.99
Settling Time (s)	515.43	263.99
Overshoot (%)	0.00	13.75
Undershoot (%)	-0.00	-0.00
Peak Time (s)	1025.12	148.79

2.4 Root Locus Technique for G22

For the transfer function G_{22} using equation [1] the current rise time was found to be ≈ 367.65 sec. Therefore, the target rise time is 122.55 sec.

Then using equations [3] and [4] it was found that:

$$\zeta \approx 0.5912$$
$$\omega_n \approx 0.01495$$

The design point was then found using [5] and [6] giving $s_d = -0.008838 + j0.0121$.

The second form of the PID equation [7] was then used to find the controller transfer function.

Furthermore, the Padé approximation [2] was used to find an approximation of the transfer function's dead time. The new plant transfer function was then found to be:

$$G_p(s) = \frac{(1 - 0.0725s)}{(1 + 0.0725s)} \frac{0.05872}{(s + 0.005984)}$$

Letting $z_2 = 0.1$ the angles of the poles and zeros to the design point, and [11], it was found:

$$\theta_{z=z_1} + \theta_{z=z_2} + \theta_{z=13.793} - (\theta_{p=0} + \theta_{p=-13.793} + \theta_{p=-0.005984}) = \pm 180^\circ$$
$$\theta_{z=z_1} \approx -180^\circ + 222.12^\circ$$
$$\theta_{z=z_1} \approx 42.119^\circ$$

Therefore, the value of z_1 was found as:

$$\tan(\theta_{z=z_1}) = \frac{0.0121}{z_1 - 0.00884}$$
$$z_1 = \frac{0.0121 + 0.00884 \tan(\theta_{z=z_1})}{\tan(\theta_{z=z_1})}$$
$$z_1 \approx 0.0221733$$

The controller gain K_c was found using [12] where:

$$|K_c G_c(s_d) G_p(s_d)| = 1$$
$$|G_c(s_d) G_p(s_d)| \approx 0.52473$$

Therefore:

$$K_c = \frac{1}{|G_c(s_d) G_p(s_d)|} \approx 1.90571$$

The final controller transfer function was then found to be:

$$G_c(s) = \frac{1.90571(s + 0.1)(s + 0.0221733)}{s}$$

Using [10], [9] and [8] the PID values where found to be:

$$K_p = K_c(z_1 + z_2) = 0.232827$$
$$K_i = K_c(z_1 z_2) = 0.0042256$$
$$K_d = K_c = 1.90571$$

In order to confirm the design, the root locus was made using matlab, where the root locus plot has an intersection very close to the design point.

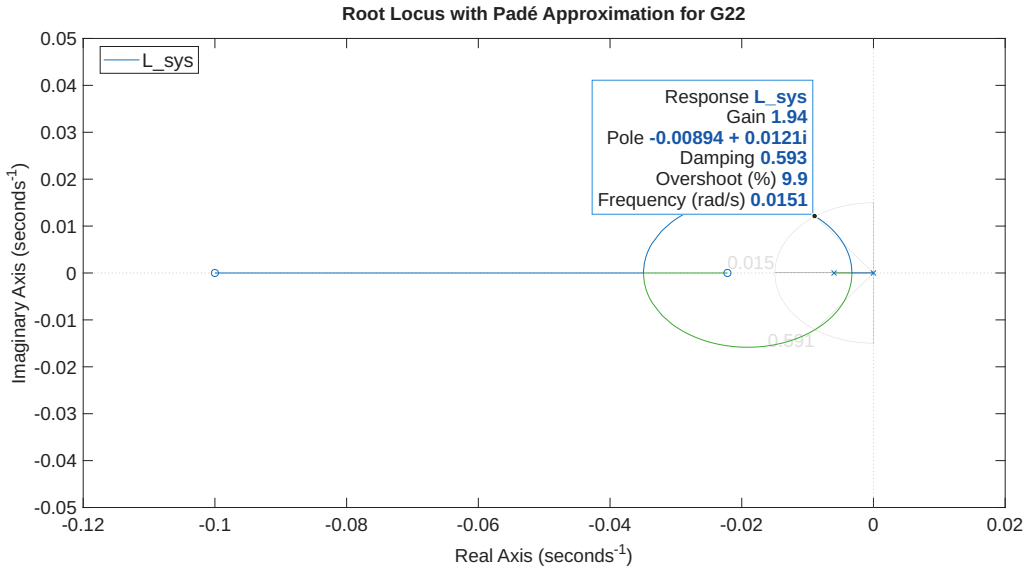


Figure 7: Root Locus plot of G22 with PID controller showing the intersection with the design point.

The resulting time domain comparison of the new closed loop transfer function and the open loop transfer function are shown below:

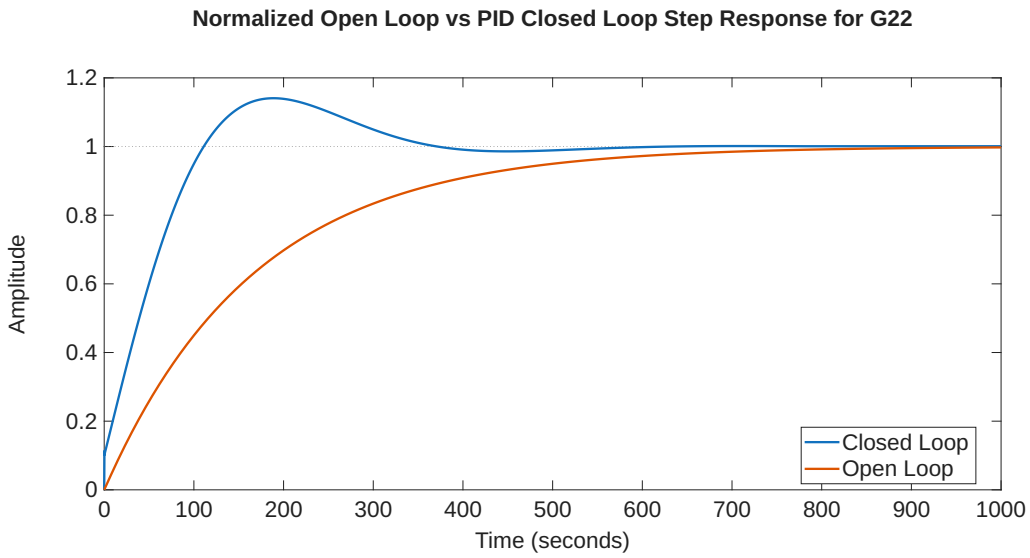


Figure 8: Time domain comparison of open loop plant transfer function against the closed loop system transfer function for G22

The following table compares the open loop and closed loop step response information.

Table 5: Table comparing open vs closed loop step response characteristics of G22

Step Response Parameter	Open Loop	Closed Loop
Rise Time (s)	367.15	90.64
Transient Time (s)	653.90	335.60
Settling Time (s)	653.90	335.60
Overshoot (%)	0.00	14.07
Undershoot (%)	-0.00	-0.00
Peak Time (s)	1300.59	188.79

2.5 Simulation Results Discussion

2.5.1 Comparison of Controllers Designed for $G_{11}(s)$

Analysing the results for the controller design for G_{11} shows how the frequency response technique yielded the worse theoretical step response characteristics. With the frequency response having an overshoot that is 7.26% over the desired 10%, and the root locus technique having an overshoot that is 3.75% over the desired 10%.

Similarly, even though both controller designs have a rise time that is less than the desired 96.59 sec, the root locus design had a rise time that is 25.50 sec faster, while the frequency response design had a rise time that is 11.30 sec faster. The root locus design also has a 0.00% undershoot, while the frequency response design has a undershoot of 8.86%.

As seen above, even though both controller designs meet the rise time requirements, they both fail in exactly reaching/exceeding the overshoot specification. This is due

to both design techniques using second order approximations to find design points, while the actual system has a higher order due to the pole and zeros introduced by the PID controller. The effect of the PID controller zeros is easily seen on the root locus plot 5 where the zero $z_1 = 0.0295055$ is only $0.0295055 - 0.01121 = 0.0182955$ away from the real component of the design point.

Table 6: Table comparing controller design parameters for different PID controller designs.

Step Response Parameter	Open Loop	Frequency Response Technique	Root Locus Technique
Rise Time (s)	289.08	85.29	71.09
Transient Time (s)	514.89	432.80	263.99
Settling Time (s)	514.89	441.73	263.99
Overshoot (%)	0.00	17.26	13.75
Undershoot (%)	-0.00	8.86	-0.00
Peak Time (s)	963.45	176.04	148.79

2.5.2 Comparison of Controllers Designed for $G_{22}(s)$

Analysing the results for the controller design for G_{22} shows how the frequency response technique again yielded the worse theoretical step response characteristics than the root locus technique. With the frequency response design having an overshoot that is 7.10% over the desired 10%, and the root locus design having an overshoot that is 4.07% over the desired 10%.

Again, even though both controller designs have a rise time that is less than the desired 122.55 sec, the root locus design had a rise time that is 31.91 sec faster, while the frequency response design had a rise time that is 15.08 sec faster. The root locus design also has a 0.00% undershoot, while the frequency response design has a undershoot of 7.40%.

Table 7: Table comparing controller design parameters for different PID controller designs.

Step Response Parameter	Open Loop	Frequency Response Technique	Root Locus Technique
Rise Time (s)	367.15	107.47	90.64
Transient Time (s)	653.90	548.75	335.60
Settling Time (s)	653.90	559.16	335.60
Overshoot (%)	0.00	17.10	14.07
Undershoot (%)	-0.00	7.40	-0.00
Peak Time (s)	1300.59	224.25	188.79

3 Results

3.1 Root Locus Designs

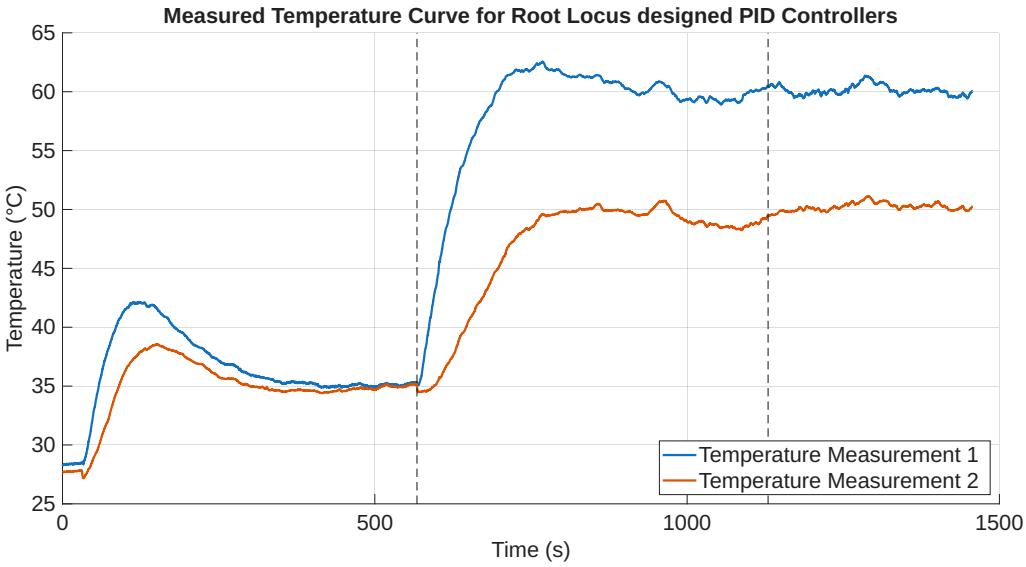


Figure 9: Measured Temperature results for the root locus designed controllers, with state transitions shown using dotted vertical lines

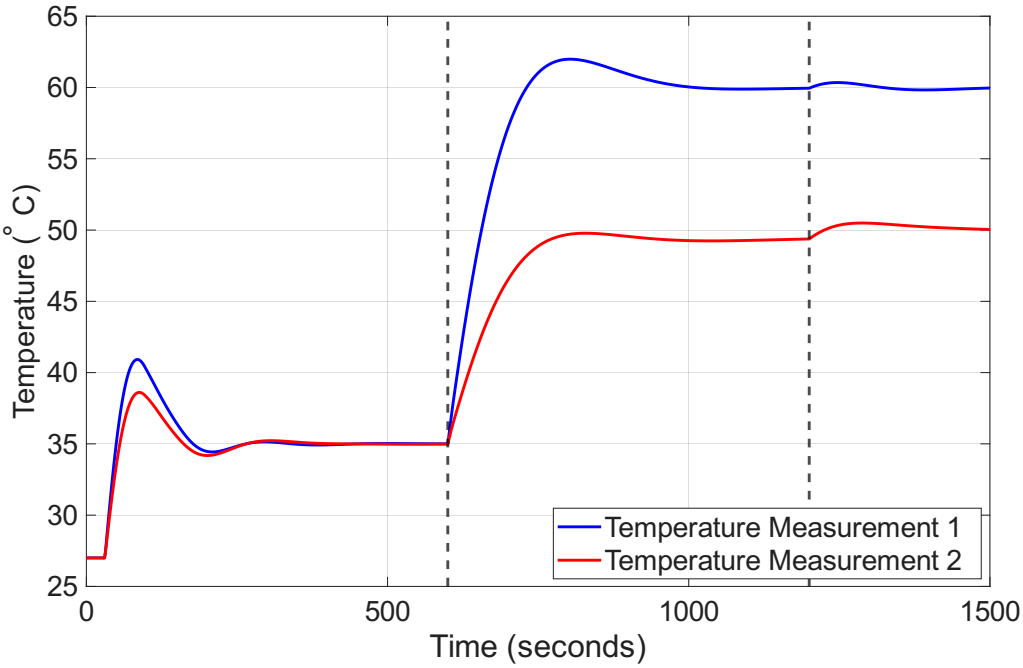


Figure 10: Measured simulated temperature results for the root locus designed controllers, with state transitions shown using dotted vertical lines

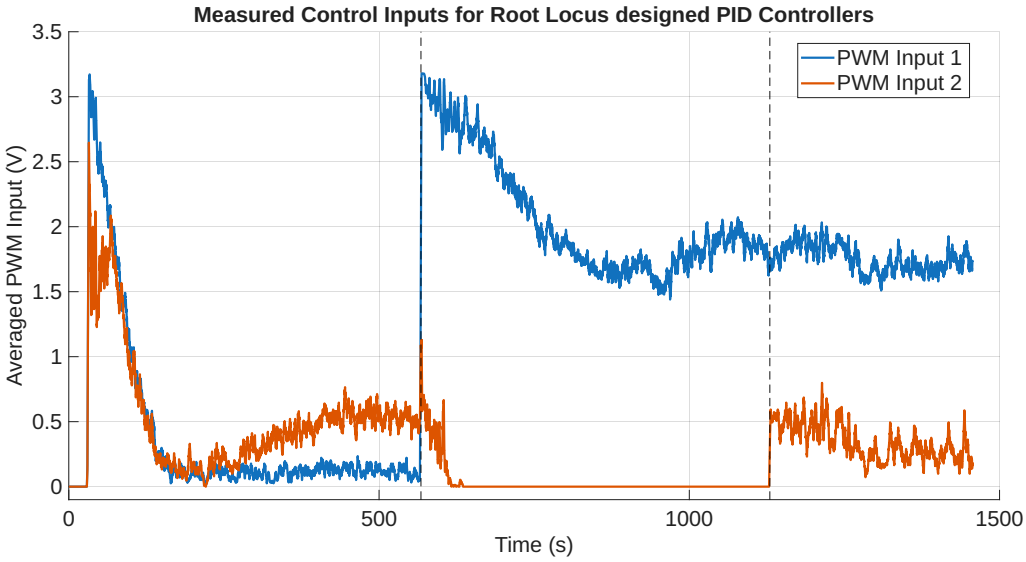


Figure 11: Measured control input results for the root locus designed controllers, with state transitions shown using dotted vertical lines

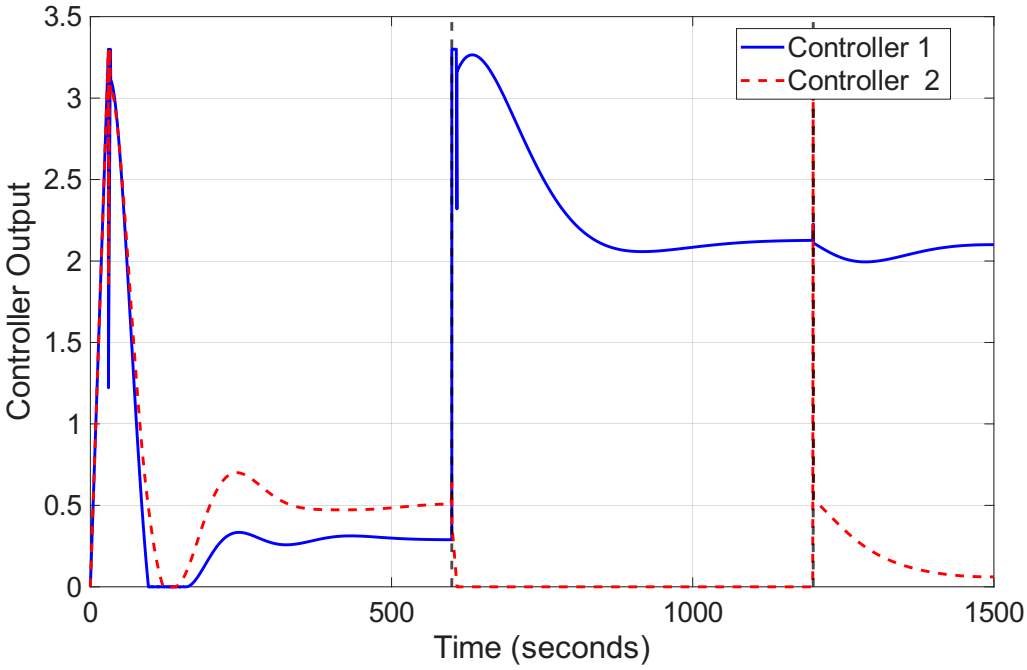


Figure 12: Measured simulated control input results for the root locus designed controllers, with state transitions shown using dotted vertical lines

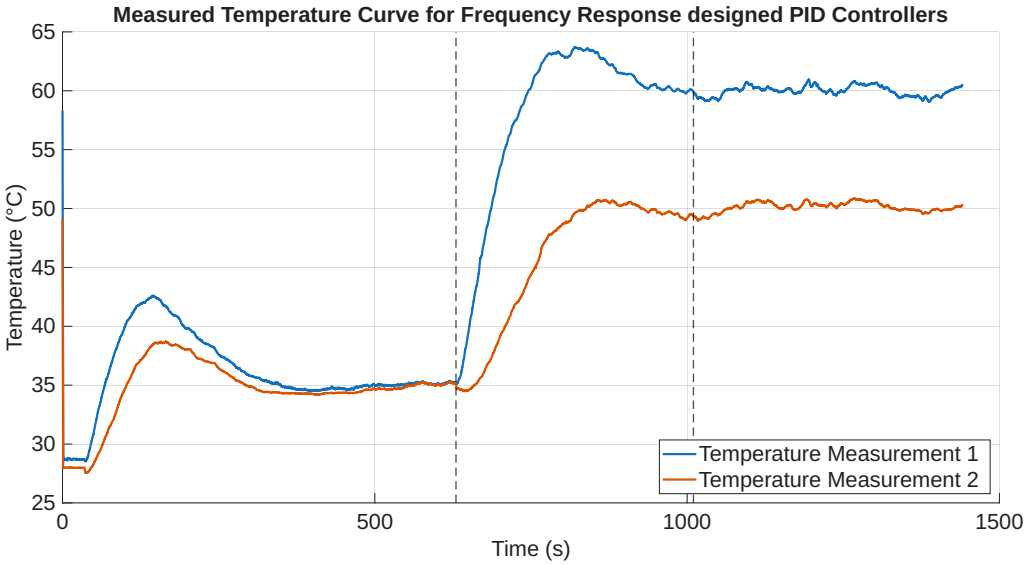


Figure 13: Measured Temperature results for the frequency response designed controllers, with state transitions shown using dotted vertical lines

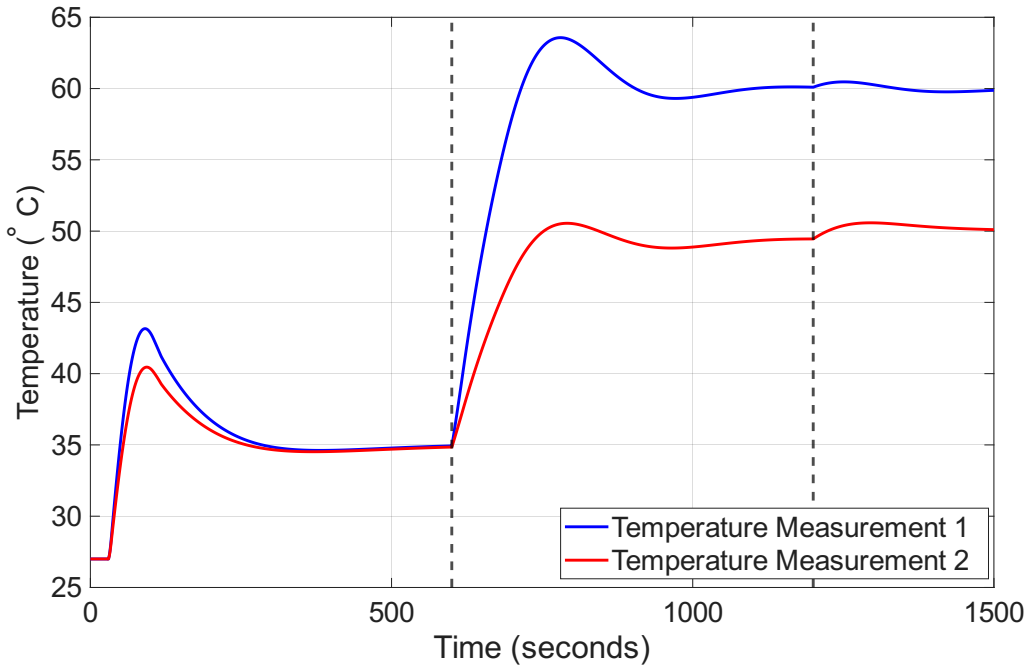


Figure 14: Measured simulated temperature results for the frequency response designed controllers, with state transitions shown using dotted vertical lines

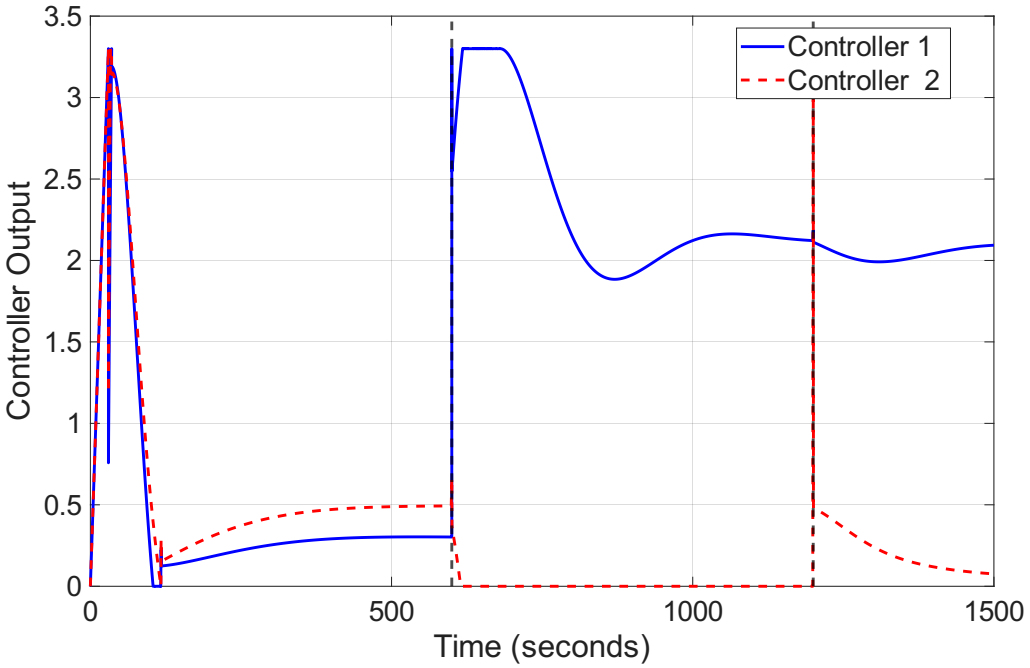


Figure 15: Measured simulated control input results for the frequency response designed controllers, with state transitions shown using dotted vertical lines

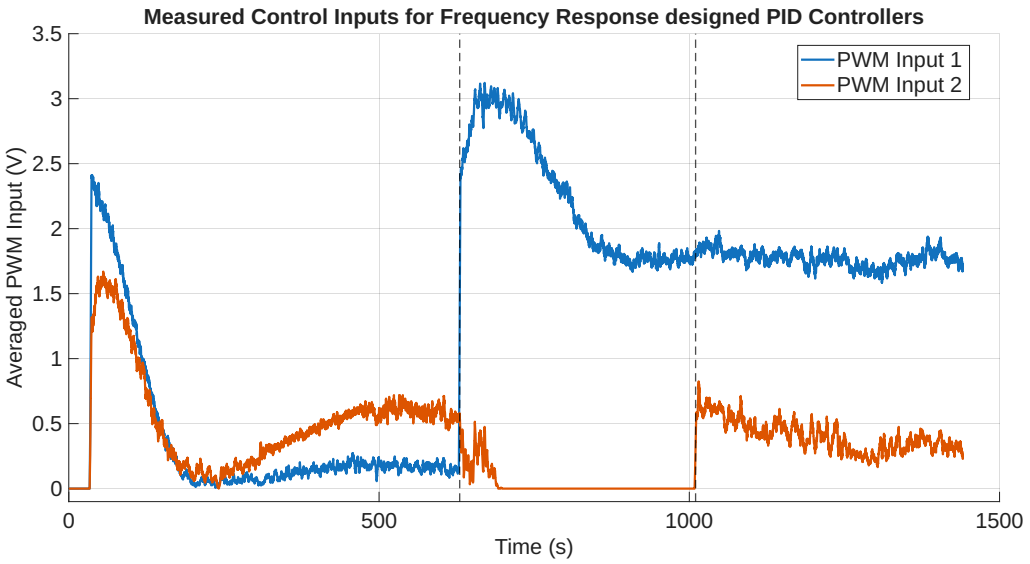


Figure 16: Measured control input results for the frequency response designed controllers, with state transitions shown using dotted vertical lines

4 Discussion

4.1 Frequency Response Technique

Table 8: Comparison of Simulated vs. Measured Transient Performance for Frequency Response PID Controllers

	Overshoot (%)		**Rise Time (s)**	
State Transition	**Simulated**	**Measured**	**Simulated**	**Measured**
State 1 (G_{11})	23.54	21.14	13.91	22.18
State 1 (G_{22})	16.09	10.43	18.27	43.85
State 2 (G_{11})	5.78	6.17	88.89	82
State 3 (G_{22})	0.95	1.68	18.63	37.63

Table 8 compares the overshoot and rise time of the simulated Simulink model against the measured physical system (Figure 13 and 14). Overall, the physical implementation exhibited less overshoot and a noticeably longer rise time. This difference primarily occurs because the plant transfer function approximates a non-linear thermodynamic process as a perfectly linear system. Unmodeled environmental variables, such as ambient laboratory temperature shifts or constant convective cooling from air conditioning drafts, act as continuous thermal disturbances. Because real-world heat loss is exponential rather than linear, the physical environment drains heat much faster than the linear model predicts. This acts as physical damping that slows the rise time and suppresses the expected overshoot.

However, as seen in Figure 16 and Figure 15, the measured steady-state control action sits at a lower average voltage than the simulation predicted. This indicates that the physical system is more efficient at maintaining its temperature once settled. This discrepancy is likely due to the linear model underestimating the plant's true DC gain during system identification, or a warmer ambient laboratory temperature during testing, which reduces the total electrical power required to hold a specific setpoint compared to the model's baseline assumption.

4.2 Root Locus Technique

Table 9: Comparison of Simulated vs. Measured Transient Performance for Root Locus PID Controllers

	Overshoot (%)		**Rise Time (s)**	
State Transition	**Simulated**	**Measured**	**Simulated**	**Measured**
State 1 (G_{11})	16.90	20.29	14.56	18.17
State 1 (G_{22})	10.35	10.15	19.59	37.59
State 2 (G_{11})	3.40	3.33	95.08	90.05
State 3 (G_{22})	0.90	2.0	19.72	37.00

4.3 Comparison of PID Coefficients Between Design Methods

Table 10 summarises the PID coefficients obtained for G_{11} and G_{22} using both design methods.

Table 10: Comparison of PID coefficients across design methods

	G_{11} **(PWM1 → T1)**		G_{22} **(PWM2 → T2)**	
Coefficient	**Freq. Response**	**Root Locus**	**Freq. Response**	**Root Locus**
K_p	0.0947	0.1527	0.1488	0.2328
K_i	0.00257	0.003479	0.00314	0.004226
K_d	0.6901	1.17921	1.174	1.90571

A consistent pattern is visible across both plants: the root locus method produces larger K_p, K_i, and K_d values than the frequency response method. Since $K_d = K_c$ and both $K_p = K_c(z_1 + z_2)$ and $K_i = K_c z_1 z_2$ scale directly with K_c, this difference originates in how each method determines K_c. The root locus method enforces the magnitude condition at the complex design point s_d, where the plant has lower magnitude, requiring a larger K_c. The frequency response method enforces the same condition at the real frequency $\omega_s = 0.8\omega_{BW}$, where the plant magnitude is higher, so a smaller K_c suffices. For G_{22} this gives $K_c = 1.906$ (root locus) versus $K_c = 1.174$ (frequency response), a factor of 1.6×.

The D-action zeros also differ slightly between the two methods , $z_1 = 0.02674$ (frequency response) versus $z_1 = 0.02217$ (root locus) for G_{22} , further amplifying the coefficient differences through the K_p and K_i expressions. Despite these numerical differences, both controller pairs met the design specifications and performed comparably in the physical implementation, confirming that the plant is robust to this level of gain variation between the two design methods.

References

[1] N. S. Nise, *Control Systems Engineering*, 8th. John Wiley & Sons, 2019, ISBN: 978-1-119-47421-0.

List of Tables

1 2x2 Process Transfer Function Matrix 4
2 Table comparing open vs closed loop step response characteristics of G11 6
3 Table comparing open vs closed loop step response characteristics of G22 9
4 Table comparing open vs closed loop step response characteristics of G11 12
5 Table comparing open vs closed loop step response characteristics of G22 14
6 Table comparing controller design parameters for different PID controller designs. 15
7 Table comparing controller design parameters for different PID controller designs. 15
8 Comparison of Simulated vs. Measured Transient Performance for Frequency Response PID Controllers 20
9 Comparison of Simulated vs. Measured Transient Performance for Root Locus PID Controllers 21
10 Comparison of PID coefficients across design methods 21

List of Figures

1 G11 Open-Loop Bode Plot verifying the 58.6° Phase Margin at 0.0177 rad/s. 5
2 G11 Normalized Open-Loop vs Closed-Loop Step Response. 6
3 G22 Open-Loop Bode Plot verifying the 58.5° Phase Margin at 0.0139 rad/s. 8
4 G22 Normalized Open-Loop vs Closed-Loop Step Response. 8
5 Root Locus plot of G11 with PID controller showing the intersection with the design point. 11
6 Time domain comparison of open loop plant transfer function against the closed loop system transfer function for G11 11
7 Root Locus plot of G22 with PID controller showing the intersection with the design point. 13
8 Time domain comparison of open loop plant transfer function against the closed loop system transfer function for G22 14
9 Measured Temperature results for the root locus designed controllers, with state transitions shown using dotted vertical lines 16
10 Measured simulated temperature results for the root locus designed controllers, with state transitions shown using dotted vertical lines . . 16
11 Measured control input results for the root locus designed controllers, with state transitions shown using dotted vertical lines 17

12 Measured simulated control input results for the root locus designed controllers, with state transitions shown using dotted vertical lines . . 17
13 Measured Temperature results for the frequency response designed controllers, with state transitions shown using dotted vertical lines . . 18
14 Measured simulated temperature results for the frequency response designed controllers, with state transitions shown using dotted vertical lines 18
15 Measured simulated control input results for the frequency response designed controllers, with state transitions shown using dotted vertical lines 19
16 Measured control input results for the frequency response designed controllers, with state transitions shown using dotted vertical lines . . 19

A Code

A.1 Matlab Code

Listing 1: App Code For Sampling

```
%
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%% Auto Generated Code Start
    %%%%%%%%%%%%%%%%%
%
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%


classdef EBBPrac < matlab.apps.AppBase

    % Properties that correspond to app components
    properties (Access = public)
        UIFigure                    matlab.ui.Figure
        GridLayout                  matlab.ui.container.
           GridLayout
        LeftPanel                   matlab.ui.container.
           Panel
        TabGroup                    matlab.ui.container.
           TabGroup
        Tab                         matlab.ui.container.
           Tab
        DataSaveEditField           matlab.ui.control.
           EditField
        DataSaveEditFieldLabel      matlab.ui.control.
           Label
        SamplingtoofastLabel        matlab.ui.control.
           Label
        UpdateChannelsButton        matlab.ui.control.
           Button
        StartButton                 matlab.ui.control.
           StateButton
        StopButton                  matlab.ui.control.
           StateButton
        OutputDropDown              matlab.ui.control.
           DropDown
        OutputDropDownLabel         matlab.ui.control.
           Label
        DutyCycleEditField          matlab.ui.control.
           NumericEditField
        DutyCycleEditFieldLabel     matlab.ui.control.
           Label
        DisturbanceEditField        matlab.ui.control.
```

```
        NumericEditField
    DisturbanceEditFieldLabel   matlab.ui.control.
        Label
    SetpointEditField           matlab.ui.control.
        NumericEditField
    SetpointEditFieldLabel      matlab.ui.control.
        Label
    DEditField                  matlab.ui.control.
        NumericEditField
    DEditFieldLabel             matlab.ui.control.
        Label
    IEditField                  matlab.ui.control.
        NumericEditField
    IEditFieldLabel             matlab.ui.control.
        Label
    PEditField                  matlab.ui.control.
        NumericEditField
    PEditFieldLabel             matlab.ui.control.
        Label
    Tab2                        matlab.ui.container.
        Tab
    BaudRateEditField           matlab.ui.control.
        NumericEditField
    BaudRateEditFieldLabel      matlab.ui.control.
        Label
    COMPortEditField            matlab.ui.control.
        EditField
    COMPortEditFieldLabel       matlab.ui.control.
        Label
    RightPanel                  matlab.ui.container.
        Panel
    UIAxes_2                    matlab.ui.control.
        UIAxes
    UIAxes                      matlab.ui.control.
        UIAxes
end

% Properties that correspond to apps with auto-reflow
properties (Access = private)
    onePanelWidth = 576;
end
%
   %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%% Auto Generated Code End
   %%%%%%%%%%%%%%%
%
   %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
properties (Access = private)
    %Live lines 1 and 2 are for the temperature
       measurements.
    liveLine1
    liveLine2
    %Live lines 3 and 4 are for the PWM signals.
    liveLine3
    liveLine4
    P
    I
    D
    setpoint
    disturbance
    duty_cycle
    output
    capturing
    COM_port
    baud_rate
    window_size
    esp32
    data_save_name % Description
end


% Callbacks that handle component events
methods (Access = private)

    % Value changed function: PEditField
    function PEditFieldValueChanged(app, event)
        app.P = app.PEditField.Value;
    end

    % Value changed function: IEditField
    function IEditFieldValueChanged(app, event)
        app.I = app.IEditField.Value;
    end

    % Value changed function: DEditField
    function DEditFieldValueChanged(app, event)
        app.D = app.DEditField.Value;
    end

    % Value changed function: SetpointEditField
    function SetpointEditFieldValueChanged(app, event
       )
        app.setpoint = app.SetpointEditField.Value;
```

```
end

% Value changed function: DisturbanceEditField
function DisturbanceEditFieldValueChanged(app,
    event)
    app.disturbance = app.DisturbanceEditField.
        Value;
end

% Value changed function: DutyCycleEditField
function DutyCycleEditFieldValueChanged(app,
    event)
    app.duty_cycle = app.DutyCycleEditField.Value
        ;
end

% Value changed function: OutputDropDown
function OutputDropDownValueChanged(app, event)
    app.output = str2double(app.OutputDropDown.
        Value);
end

% Value changed function: StartButton
function StartButtonValueChanged(app, event)
    app.StartButton.Visible = "off";
    app.StopButton.Visible = "on";
    app.UpdateChannelsButton.Visible = "on";
    app.capturing = true;

    %Call all of the other functions to make sure
        their values are
    %captured.
    app.PEditFieldValueChanged();
    app.IEditFieldValueChanged();
    app.DEditFieldValueChanged();
    app.SetpointEditFieldValueChanged();
    app.DisturbanceEditFieldValueChanged();
    app.DutyCycleEditFieldValueChanged();
    app.OutputDropDownValueChanged();
    app.BaudRateEditFieldValueChanged();
    app.COMPortEditFieldValueChanged();
    app.DataSaveEditFieldValueChanged();

    if isempty(app.esp32)
        app.esp32 = serialport(app.COM_port, app.
            baud_rate);
        configureTerminator(app.esp32, "LF");
    end
```

```
flush(app.esp32);

pause(1);

%Make sure all info is read into esp.
writeline(app.esp32, sprintf("%d, %.2f", 0,
   app.P)); %
writeline(app.esp32, sprintf("%d, %.2f", 1,
   app.I)); %
writeline(app.esp32, sprintf("%d, %.2f", 2,
   app.D)); %
writeline(app.esp32, sprintf("%d, %.2f", 3,
   app.setpoint)); %
writeline(app.esp32, sprintf("%d, %.2f", 4,
   app.disturbance)); %
writeline(app.esp32, sprintf("%d, %d", 5, app
   .duty_cycle)); %
writeline(app.esp32, sprintf("%d, %.2f", 6,
   app.output)); %
writeline(app.esp32, sprintf("%d, %.2f", 7,
   app.capturing)); %

%Clear axes.
cla(app.UIAxes);
cla(app.UIAxes_2);

%Plotting axes for the temperature
   measurements.
app.liveLine1 = animatedline(app.UIAxes, '
   Color', 'r');
app.liveLine2 = animatedline(app.UIAxes, '
   Color', 'b');

%Plotting axes for the PWM signals.
app.liveLine3 = animatedline(app.UIAxes_2, '
   Color', 'r');
app.liveLine4 = animatedline(app.UIAxes_2, '
   Color', 'b');

app.window_size = 30; %seconds

%Define data csv that we will safe to.
fid = fopen(app.data_save_name + ".csv", 'wt'
   );

while(app.capturing)
    if app.esp32.NumBytesAvailable > 0
        rawData = readline(app.esp32);
```

```
                data = str2double(split(rawData, ',')
                    );
                data(5) = data(5)/1000;

                if ~isnan(data(1)) && (length(data)
                    == 6)
                    %Plot temperature data.
                    addpoints(app.liveLine1, data(5),
                        data(1));
                    addpoints(app.liveLine2, data(5),
                        data(2));
                    xlim(app.UIAxes, [data(5) - app.
                        window_size, data(5)]);

                    %Plot PWM signals.
                    addpoints(app.liveLine3, data(5),
                        data(3));
                    addpoints(app.liveLine4, data(5),
                        data(4));
                    xlim(app.UIAxes_2, [data(5) - app
                        .window_size, data(5)]);

                    %Write data to file.
                    fprintf(fid, rawData);

                    if data(end) == 0
                        app.SamplingtoofastLabel.
                            Visible = "on";
                    else
                        app.SamplingtoofastLabel.
                            Visible = "off";
                    end
                end

                drawnow limitrate
            end
        end

        %Close file we are saving to.
        fclose(fid);

        %Send stop signal.
        writeline(app.esp32, sprintf("%d, %.2f", 7,
            app.capturing));

        clear app.esp32;
    end
```

```
% Value changed function: COMPortEditField
function COMPortEditFieldValueChanged(app, event)
    app.COM_port = app.COMPortEditField.Value;
end

% Value changed function: BaudRateEditField
function BaudRateEditFieldValueChanged(app, event
    )
    app.baud_rate = app.BaudRateEditField.Value;
end

% Value changed function: StopButton
function StopButtonValueChanged(app, event)
    app.capturing = false;
    app.StopButton.Visible = "off";
    app.UpdateChannelsButton.Visible = "off";
    app.StartButton.Visible = "on";

end

% Button pushed function: UpdateChannelsButton
function UpdateChannelsButtonPushed(app, event)
    app.DutyCycleEditFieldValueChanged();
    app.OutputDropDownValueChanged();
    writeline(app.esp32, sprintf("%d, %.2f", 5,
        app.duty_cycle));
    writeline(app.esp32, sprintf("%d, %.2f", 6,
        app.output));
end

% Value changed function: DataSaveEditField
function DataSaveEditFieldValueChanged(app, event
    )
    app.data_save_name = app.DataSaveEditField.
        Value;

end

% Changes arrangement of the app based on
    UIFigure width
function updateAppLayout(app, event)
    currentFigureWidth = app.UIFigure.Position(3)
        ;
    if(currentFigureWidth <= app.onePanelWidth)
        % Change to a 2x1 grid
        app.GridLayout.RowHeight = {449, 449};
        app.GridLayout.ColumnWidth = {'1x'};
        app.RightPanel.Layout.Row = 2;
```

```
            app.RightPanel.Layout.Column = 1;
        else
            % Change to a 1x2 grid
            app.GridLayout.RowHeight = {'1x'};
            app.GridLayout.ColumnWidth = {204, '1x'};
            app.RightPanel.Layout.Row = 1;
            app.RightPanel.Layout.Column = 2;
        end
    end
end

%
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%% Auto Generated Code Start
    %%%%%%%%%%%%%
%
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%


% Component initialization
methods (Access = private)

    % Create UIFigure and components
    function createComponents(app)

        % Create UIFigure and hide until all
           components are created
        app.UIFigure = uifigure('Visible', 'off');
        app.UIFigure.AutoResizeChildren = 'off';
        app.UIFigure.Position = [100 100 702 449];
        app.UIFigure.Name = 'MATLAB App';
        app.UIFigure.SizeChangedFcn =
           createCallbackFcn(app, @updateAppLayout,
           true);

        % Create GridLayout
        app.GridLayout = uigridlayout(app.UIFigure);
        app.GridLayout.ColumnWidth = {204, '1x'};
        app.GridLayout.RowHeight = {'1x'};
        app.GridLayout.ColumnSpacing = 0;
        app.GridLayout.RowSpacing = 0;
        app.GridLayout.Padding = [0 0 0 0];
        app.GridLayout.Scrollable = 'on';

        % Create LeftPanel
        app.LeftPanel = uipanel(app.GridLayout);
        app.LeftPanel.Layout.Row = 1;
```

```
app.LeftPanel.Layout.Column = 1;

% Create TabGroup
app.TabGroup = uitabgroup(app.LeftPanel);
app.TabGroup.Position = [3 3 197 442];

% Create Tab
app.Tab = uitab(app.TabGroup);
app.Tab.Title = 'Tab';

% Create PEditFieldLabel
app.PEditFieldLabel = uilabel(app.Tab);
app.PEditFieldLabel.HorizontalAlignment = '
   right';
app.PEditFieldLabel.Position = [46 385 25
   22];
app.PEditFieldLabel.Text = 'P';

% Create PEditField
app.PEditField = uieditfield(app.Tab, '
   numeric');
app.PEditField.ValueChangedFcn =
   createCallbackFcn(app, @
   PEditFieldValueChanged, true);
app.PEditField.Position = [85 385 100 22];

% Create IEditFieldLabel
app.IEditFieldLabel = uilabel(app.Tab);
app.IEditFieldLabel.HorizontalAlignment = '
   right';
app.IEditFieldLabel.Position = [46 355 25
   22];
app.IEditFieldLabel.Text = 'I';

% Create IEditField
app.IEditField = uieditfield(app.Tab, '
   numeric');
app.IEditField.ValueChangedFcn =
   createCallbackFcn(app, @
   IEditFieldValueChanged, true);
app.IEditField.Position = [85 355 100 22];

% Create DEditFieldLabel
app.DEditFieldLabel = uilabel(app.Tab);
app.DEditFieldLabel.HorizontalAlignment = '
   right';
app.DEditFieldLabel.Position = [46 326 25
   22];
```

```
app.DEditFieldLabel.Text = 'D';

% Create DEditField
app.DEditField = uieditfield(app.Tab, '
   numeric');
app.DEditField.ValueChangedFcn =
   createCallbackFcn(app, @
   DEditFieldValueChanged, true);
app.DEditField.Position = [85 326 100 22];

% Create SetpointEditFieldLabel
app.SetpointEditFieldLabel = uilabel(app.Tab)
   ;
app.SetpointEditFieldLabel.
   HorizontalAlignment = 'right';
app.SetpointEditFieldLabel.Position = [21 296
    49 22];
app.SetpointEditFieldLabel.Text = 'Setpoint';

% Create SetpointEditField
app.SetpointEditField = uieditfield(app.Tab,
   'numeric');
app.SetpointEditField.ValueChangedFcn =
   createCallbackFcn(app, @
   SetpointEditFieldValueChanged, true);
app.SetpointEditField.Position = [85 296 100
   22];

% Create DisturbanceEditFieldLabel
app.DisturbanceEditFieldLabel = uilabel(app.
   Tab);
app.DisturbanceEditFieldLabel.
   HorizontalAlignment = 'right';
app.DisturbanceEditFieldLabel.Position = [1
   265 69 22];
app.DisturbanceEditFieldLabel.Text = '
   Disturbance';

% Create DisturbanceEditField
app.DisturbanceEditField = uieditfield(app.
   Tab, 'numeric');
app.DisturbanceEditField.ValueChangedFcn =
   createCallbackFcn(app, @
   DisturbanceEditFieldValueChanged, true);
app.DisturbanceEditField.Position = [85 265
   100 22];

% Create DutyCycleEditFieldLabel
```

```
app.DutyCycleEditFieldLabel = uilabel(app.Tab
    );
app.DutyCycleEditFieldLabel.
    HorizontalAlignment = 'right';
app.DutyCycleEditFieldLabel.Position = [9 235
     63 22];
app.DutyCycleEditFieldLabel.Text = 'Duty
    Cycle';

% Create DutyCycleEditField
app.DutyCycleEditField = uieditfield(app.Tab,
     'numeric');
app.DutyCycleEditField.ValueChangedFcn =
    createCallbackFcn(app, @
    DutyCycleEditFieldValueChanged, true);
app.DutyCycleEditField.Position = [86 235 100
     22];
app.DutyCycleEditField.Value = 100;

% Create OutputDropDownLabel
app.OutputDropDownLabel = uilabel(app.Tab);
app.OutputDropDownLabel.HorizontalAlignment =
     'right';
app.OutputDropDownLabel.Position = [30 199 41
     22];
app.OutputDropDownLabel.Text = 'Output';

% Create OutputDropDown
app.OutputDropDown = uidropdown(app.Tab);
app.OutputDropDown.Items = {'None', 'Channel1
    ', 'Channel2', 'Both'};
app.OutputDropDown.ItemsData = {'3', '0', '1'
    , '2'};
app.OutputDropDown.ValueChangedFcn =
    createCallbackFcn(app, @
    OutputDropDownValueChanged, true);
app.OutputDropDown.Position = [85 199 100
    22];
app.OutputDropDown.Value = '3';

% Create StopButton
app.StopButton = uibutton(app.Tab, 'state');
app.StopButton.ValueChangedFcn =
    createCallbackFcn(app, @
    StopButtonValueChanged, true);
app.StopButton.Visible = 'off';
app.StopButton.Text = 'Stop';
app.StopButton.Position = [85 168 100 23];
```

```
% Create StartButton
app.StartButton = uibutton(app.Tab, 'state');
app.StartButton.ValueChangedFcn =
   createCallbackFcn(app, @
   StartButtonValueChanged, true);
app.StartButton.Text = 'Start';
app.StartButton.Position = [86 168 100 23];

% Create UpdateChannelsButton
app.UpdateChannelsButton = uibutton(app.Tab,
   'push');
app.UpdateChannelsButton.ButtonPushedFcn =
   createCallbackFcn(app, @
   UpdateChannelsButtonPushed, true);
app.UpdateChannelsButton.Visible = 'off';
app.UpdateChannelsButton.Position = [81 136
   108 23];
app.UpdateChannelsButton.Text = 'Update
   Channels';

% Create SamplingtoofastLabel
app.SamplingtoofastLabel = uilabel(app.Tab);
app.SamplingtoofastLabel.Visible = 'off';
app.SamplingtoofastLabel.Position = [90 12
   101 22];
app.SamplingtoofastLabel.Text = 'Sampling too
    fast!';

% Create DataSaveEditFieldLabel
app.DataSaveEditFieldLabel = uilabel(app.Tab)
   ;
app.DataSaveEditFieldLabel.
   HorizontalAlignment = 'right';
app.DataSaveEditFieldLabel.Position = [10 96
   61 22];
app.DataSaveEditFieldLabel.Text = 'Data Save'
   ;

% Create DataSaveEditField
app.DataSaveEditField = uieditfield(app.Tab,
   'text');
app.DataSaveEditField.ValueChangedFcn =
   createCallbackFcn(app, @
   DataSaveEditFieldValueChanged, true);
app.DataSaveEditField.Position = [86 96 100
   22];
app.DataSaveEditField.Value = 'PWM1StepV0';
```

```
% Create Tab2
app.Tab2 = uitab(app.TabGroup);
app.Tab2.Title = 'Tab2';

% Create COMPortEditFieldLabel
app.COMPortEditFieldLabel = uilabel(app.Tab2)
    ;
app.COMPortEditFieldLabel.HorizontalAlignment
     = 'right';
app.COMPortEditFieldLabel.Position = [13 385
    58 22];
app.COMPortEditFieldLabel.Text = 'COM Port';

% Create COMPortEditField
app.COMPortEditField = uieditfield(app.Tab2,
    'text');
app.COMPortEditField.ValueChangedFcn =
    createCallbackFcn(app, @
    COMPortEditFieldValueChanged, true);
app.COMPortEditField.Position = [85 385 100
    22];
app.COMPortEditField.Value = '/dev/ttyUSB0';

% Create BaudRateEditFieldLabel
app.BaudRateEditFieldLabel = uilabel(app.Tab2
    );
app.BaudRateEditFieldLabel.
    HorizontalAlignment = 'right';
app.BaudRateEditFieldLabel.Position = [9 355
    62 22];
app.BaudRateEditFieldLabel.Text = 'Baud Rate'
    ;

% Create BaudRateEditField
app.BaudRateEditField = uieditfield(app.Tab2,
     'numeric');
app.BaudRateEditField.ValueChangedFcn =
    createCallbackFcn(app, @
    BaudRateEditFieldValueChanged, true);
app.BaudRateEditField.Position = [85 355 100
    22];
app.BaudRateEditField.Value = 115200;

% Create RightPanel
app.RightPanel = uipanel(app.GridLayout);
app.RightPanel.Layout.Row = 1;
app.RightPanel.Layout.Column = 2;
```

```
            % Create UIAxes
            app.UIAxes = uiaxes(app.RightPanel);
            title(app.UIAxes, 'Title')
            xlabel(app.UIAxes, 'X')
            ylabel(app.UIAxes, 'Y')
            zlabel(app.UIAxes, 'Z')
            app.UIAxes.Position = [7 213 489 235];

            % Create UIAxes_2
            app.UIAxes_2 = uiaxes(app.RightPanel);
            title(app.UIAxes_2, 'Title')
            xlabel(app.UIAxes_2, 'X')
            ylabel(app.UIAxes_2, 'Y')
            zlabel(app.UIAxes_2, 'Z')
            app.UIAxes_2.Position = [8 7 488 207];

            % Show the figure after all components are
               created
            app.UIFigure.Visible = 'on';
        end
    end

    % App creation and deletion
    methods (Access = public)

        % Construct app
        function app = EBBPrac

            % Create UIFigure and components
            createComponents(app)

            % Register the app with App Designer
            registerApp(app, app.UIFigure)

            if nargout == 0
                clear app
            end
        end

        % Code that executes before app deletion
        function delete(app)

            % Delete UIFigure when app is deleted
            delete(app.UIFigure)
        end
    end
end
```

```
%
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%% Auto Generated Code end
    %%%%%%%%%%%%%
%
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

A.2 Model Fit Code

Listing 2: Model fitting and plotting code

```
% Obtaining the transfer function
% Package the input arrays of both transistors into one
    matrix (Needs to be columns)
PWM1StepV1Data = readmatrix('..\Data1\Step_PWM1_V0.csv');
PWM1StepV1 = PWM1StepV1Data(7920:end,1:5);
PWM2StepV0Data = readmatrix('..\Data1\Step_PWM2_V0.csv');
PWM2StepV0 = PWM2StepV0Data(6400:end,1:5);

PWM1TestDataFile = readmatrix('..\Data1\
    Step_PWM1_VtestData50.csv');
PWM1TestData = PWM1TestDataFile(6230:end,1:5);
PWM2TestDataFile = readmatrix('..\Data1\
    Step_PWM2_VtestData50.csv');
PWM2TestData = PWM2TestDataFile(6160:end,1:5);

% pwm1 data prep
V_PWM1 = [PWM1StepV1(:,3)];
Temp_PWM1 = [PWM1StepV1(:,1)-22.28, PWM1StepV1(:,2)
    -21.24];
time_step = 0.005;

% pwm2 data prep
V_PWM2 = [PWM2StepV0(:,4)];
Temp_PWM2 = [PWM2StepV0(:,1)-20.26, PWM2StepV0(:,2)
    -19.05];

% pwm1 test data prep
V_PWM1_Test = [PWM1TestData(:,3)];
Temp_PWM1_test = [PWM1TestData(:,1)-21.2, PWM1TestData
    (:,2)-20.2];

% pwm2 test data prep
V_PWM2_Test = [PWM2TestData(:,4)];
Temp_PWM2_test = [PWM2TestData(:,1)-21.1, PWM2TestData
    (:,2)-20.2] ;
```

```
% use iddata to set up the data for tfest
tfest_data1 = iddata(Temp_PWM1, V_PWM1, time_step);
tfest_data2 = iddata(Temp_PWM2, V_PWM2, time_step);
testData1 = iddata(Temp_PWM1_test, V_PWM1_Test, time_step
    );
testData2 = iddata(Temp_PWM2_test, V_PWM2_Test, time_step
    );

% estimate nr of poles and zeros
n_poles = 1;
n_zeros = 0;

% Estimate the transfer functions independently (SISO) to
    avoid idss error
% tfest_data(samples, output, input)
sys11 = tfest(tfest_data1(:, 1, 1), n_poles, n_zeros, NaN
    ); % PWM1 to T1
sys21 = tfest(tfest_data1(:, 2, 1), n_poles, n_zeros, NaN
    ); % PWM1 to T2

sys12 = tfest(tfest_data2(:, 1, 1), n_poles, n_zeros, NaN
    ); % PWM2 to T1
sys22 = tfest(tfest_data2(:, 2, 1), n_poles, n_zeros, NaN
    ); % PWM2 to T2

% Combine the individual SISO models back into SIMO
    systems
simo_transfer1 = [sys11; sys21];
simo_transfer2 = [sys12; sys22];

% set up graph details
tfest_data1.Name = 'Measured Data';
tfest_data2.Name = 'Measured Data';
testData1.Name = 'Measured Data';
testData2.Name = 'Measured Data';
simo_transfer2.Name = 'Simulated Model';
simo_transfer1.Name = 'Simulated Model';

figure(1)
compare(tfest_data1, simo_transfer1);
legend('Location', 'best');
ylabel('Change in Temperature (C)')

figure(2)
compare(tfest_data2, simo_transfer2);
legend('Location', 'best');
ylabel('Change in Temperature (C)')
```

```
figure(3)
compare(testData1, simo_transfer1);
legend('Location', 'best');
ylabel('Change in Temperature (C)')

figure(4)
compare(testData2, simo_transfer2);
legend('Location', 'best');
ylabel('Change in Temperature (C)')

lines = findobj(gcf, 'Type', 'Line');
set(lines, 'LineWidth', 1.5);

% display the transfer functions of PWM1
[num1, den1] = tfdata(simo_transfer1(1, 1), 'v');
delay1 = simo_transfer1(1, 1).IODelay; % Extract dead
   time
[num2, den2] = tfdata(simo_transfer1(2, 1), 'v');
delay2 = simo_transfer1(2, 1).IODelay;

disp('--- Transfer Function 1 (PWM1 to T1) ---');
disp('Numerator:'); disp(num1);
disp('Denominator:'); disp(den1);
disp(['Estimated Dead Time (s): ', num2str(delay1)]);

disp('--- Transfer Function 2 (PWM1 to T2) ---');
disp('Numerator:'); disp(num2);
disp('Denominator:'); disp(den2);
disp(['Estimated Dead Time (s): ', num2str(delay2)]);

% display transfer functions of PWM2
[num12, den12] = tfdata(simo_transfer2(1, 1), 'v');
delay12 = simo_transfer2(1, 1).IODelay;
[num22, den22] = tfdata(simo_transfer2(2, 1), 'v');
delay22 = simo_transfer2(2, 1).IODelay;

disp('--- Transfer Function 1 (PWM2 to T1) ---');
disp('Numerator:'); disp(num12);
disp('Denominator:'); disp(den12);
disp(['Estimated Dead Time (s): ', num2str(delay12)]);

disp('--- Transfer Function 2 (PWM2 to T2) ---');
disp('Numerator:'); disp(num22);
disp('Denominator:'); disp(den22);
disp(['Estimated Dead Time (s): ', num2str(delay22)]);
```

A.3 Prac 2 — Controller Design (P and PI)

A.4 Prac 2 — Report Figure Generation (P and PI)

A.5 MCU Code

```
#include <fmt/core.h>
#include <Arduino.h>
#include <iostream>
#include <sstream>
#include <vector>
#include <string>
#include <queue>
#include "ValuesPID.cpp"


double duty1 = 128; //50% duty cycle by default.
double duty2 = 128;
int control_outputs = 0; //By default only pwm1 is on.
bool start = 0; //Start off.
int message = 100; //Flags for certain messages.
int start_time = 0;
bool startControlling = 0; //See if we should start
    controlling, or continue just measuring.




//New PID structs for eventual use in the controllers.
ValuesPID controllerG11 = {0, 0, 0, 0, 0, 0, 0, 0};
ValuesPID controllerG22 = {0, 0, 0, 0, 0, 0, 0, 0};


double temp1 = 0; //Final temperature measurement that is
    sent to PC and used by controller/s.
double temp2 = 0; //Final temperature measurement that is
    sent to PC and used by controller/s.
int N_samples = 200; //Amount of samples being averaged.
std::queue<double> samples1; //Storage of samples for temp
    measurements for T1
std::queue<double> samples2; //Storage of samples for temp
    measurements for T2

hw_timer_t *timer = NULL; //Timer to ensure constant sample
    rate.
const int PWM_Out_1 = GPIO_NUM_32;
const int PWM_Out_2 = GPIO_NUM_25;
const int PWM_Meas_1 = GPIO_NUM_33;
const int PWM_Meas_2 = GPIO_NUM_26;
```

```
const int Temp_Meas1 = GPIO_NUM_27;
const int Temp_Meas2 = GPIO_NUM_35;

const double A = (3.3/4095.0); //Constant multiplier used to
    scale PWM outputs to voltages.
const int PWM_freq = 10;
const int PWM_res = 8;

const int BaudRate = 1152000;
const int sampling_period = 5000; //In microseconds.
const float dt = sampling_period/1000000.0; //t delta used
   in PWM calculations.
bool first_run = true; //Check if this is the very first run
   , used the prevent initial derivative kick. (Can be used
   for both controllers at once since they are controlling
   at the same time?)

float calculatePID(ValuesPID& controller){

      float duty = 0;
      float next_integral = controller.integral + controller
         .error * dt;

      controller.derivative = (controller.error - controller
         .prev_error)/dt;

      float pid_out = controller.P * controller.error +
         controller.I*next_integral + controller.D*
         controller.derivative;
      //anti-windup logic
      if  (pid_out >= 3.3 )
      {
        duty = 1;
        if ( controller.error < 0){
        controller.integral = next_integral;
        }

      }

      else if (pid_out <= 0.0 ) {
      duty = 0.0; // You cannot have a negative duty cycle
         for a simple heater/cooler
      if ( controller.error > 0){
        controller.integral = next_integral;
        }

      }
```

```
        else{
          duty = pid_out/3.3;
          controller.integral = next_integral;
        }
        duty = (pow(2, PWM_res) - 1)*(duty);

    return duty;
}

void setup() {
  //Start the USB-C serial/UART port.
  Serial.begin(BaudRate);

  //Setup I/O pins.
  pinMode(Temp_Meas1, INPUT);
  pinMode(Temp_Meas2, INPUT);
  pinMode(PWM_Meas_1, INPUT);
  pinMode(PWM_Meas_2, INPUT);

  ledcChangeFrequency(0, PWM_freq, PWM_res);
  ledcChangeFrequency(1, PWM_freq, PWM_res);
  ledcAttachPin (PWM_Out_1, 0);
  ledcAttachPin(PWM_Out_2, 1);

  timer = timerBegin(0, 80, true);

  samples1.push(0);
  samples2.push(0);
}

void loop() {

  //If receiving communication from computer, update values.
  if(Serial.available() > 0)
  {
    //Read the entire line, trim whitespace, and separate
       the comma separated line into the command value pair.
    String received = Serial.readStringUntil('\n');
    received.trim();

    std::istringstream stream(received.c_str());
    std::string command;
    std::string value_str;
    double value;

    std::getline(stream, command, ',');
    std::getline(stream, value_str, ',');
```

```
value = std::stod(value_str);

//Essentially a "switch case" to update the appropriate
    values for each command.
if(command == "0") // P value for PWM1
{
  controllerG11.P = value;
}
else if(command == "1") // I value for PWM1
{
  controllerG11.I = value;
}
else if(command == "2") // D value for PWM1
{
  controllerG11.D = value;
}
else if(command == "3") // Setpoint value for PWM1
{
  controllerG11.setpoint = value;
}

else if(command == "5") //Change from duty cycle set to
    the startControlling flag.
{
  startControlling = value;
}
else if(command == "7") //Flag to start process of
    taking measurements and sending them over UART.
{
  start = value;

  //Check if start is true, then start timer.
  if(start)
  {
    start_time = millis();
  }
}

else if(command == "8") // P value for PWM2
{
  controllerG22.P = value;
}
else if(command == "9") // I value for PWM2
{
  controllerG22.I = value;
}
else if(command == "10") // D value for PWM2
{
```

```
      controllerG22.D = value;
    }
    else if(command == "11") // Setpoint value for PWM2
    {
      controllerG22.setpoint = value;
    }
  }

  //If start flag is true, start measuring inputs/start
     controlling aswell.
  if(start)
  {
    //Check when we started in order to see how long we were
        busy.
    timerRestart(timer);

    if(startControlling)
    {
      //Turn on the appropriate PWM signals.
        ledcWrite(0, duty1);
        ledcWrite(1, duty2);
    }

    //Interpret sensor data.
    double temp1_raw = (analogReadMilliVolts(Temp_Meas1)
       /1000.0 - 0.5)/0.01;
    double temp2_raw = (analogReadMilliVolts(Temp_Meas2)
       /1000.0 - 0.5)/0.01;

    if(samples1.size() > N_samples)
    {
      temp1 -= samples1.front();
      temp2 -= samples2.front();

      samples1.pop();
      samples2.pop();
    }

    temp1 += temp1_raw/N_samples;
    temp2 += temp2_raw/N_samples;

    samples1.push(temp1_raw/N_samples);
    samples2.push(temp2_raw/N_samples);

    //Send sensor input data as a comma seperated list.
    std::string sending = fmt::format("{0},␣{1},␣{2},␣{3},␣
       {4},␣{5}",
    temp1, temp2, analogRead(PWM_Meas_1)*A, analogRead(
```

```
    PWM_Meas_2)*A, millis() - start_time, message);

Serial.println(String(sending.c_str()));
Serial.flush();


if (startControlling)
{
  //PID controller start
  controllerG11.prev_error = controllerG11.error;
  controllerG11.error = controllerG11.setpoint - temp1;

  controllerG22.prev_error =  controllerG22.error;
  controllerG22.error =  controllerG22.setpoint - temp2;

  // Prevent derivative kick on first run

  if (first_run) {
    controllerG11.prev_error = controllerG11.error;
    controllerG22.prev_error = controllerG22.error;

    first_run = false;
  }

  duty1 = calculatePID(controllerG11);
  duty2 =calculatePID(controllerG22);



  // std::string sending = fmt::format("{0}, {1}, {2},
     {3}, {4}", P, I, D, error, duty);
  // Serial.println(String(sending.c_str()));
  // Serial.flush();
}
//Otherwise turn off control signals.
else
{
  ledcWrite(0, 0);
  ledcWrite(1, 0);
}

//Check how long everything took in microseconds.
int elapsed = timerReadMicros(timer);

if(elapsed <= sampling_period)
{
  delayMicroseconds(sampling_period - elapsed);
  message = 100; //Reset message flag to show nothing.
```

```
      }
      else
      {
        delayMicroseconds(sampling_period);
        message = 0; //Set message flag for sampling too fast.
      }


    //Otherwise deactivate PWM outputs.
    }
    else
    {
      ledcWrite(0, 0);
      ledcWrite(1, 0);
    }
}
```